



MATHEMATICAL MISSION CONTROL

SYSTEM DOCUMENTATION

"Explore Math in Space" — An immersive, futuristic educational platform teaching 6 mathematical sequences and algorithms through interactive space-themed missions.

NEXT.JS 16 · TYPESCRIPT · TAILWIND CSS · ZUSTAND · FRAMER MOTION
VERSION 1.0 — 2026

Table of Contents

01	Project Overview
02	Technology Stack
03	Architecture & Design
04	Page Flow & User Journey
05	The 6 Missions (Curriculum)
06	Component Breakdown
07	State Management
08	Sound & Audio Engine
09	Math Algorithms Engine
10	Progress & Achievement System
11	Visual Design System
12	Asset Library
13	File Structure
14	Configuration
15	How to Run

1. Project Overview

Mathematical Mission Control is a single-page web application that transforms mathematical education into an interstellar adventure. Users assume the role of space cadets who log into a futuristic mission control center, select mathematical "missions" (each represented as a planet), launch rockets to those planets, and learn through interactive simulations, visualizations, and quizzes.

Key Features

- **Immersive Login Experience** — Sci-fi themed authentication with radar animations, telemetry panels, scanline effects, and background music
- **Rocket Launch Animations** — Multi-phase countdown-to-landing sequences with particle effects, screen shake, and planet-specific landing visuals
- **6 Interactive Missions** — Each mission covers a mathematical sequence/algorithm with 8 learning tabs
- **Real-time Math Simulator** — Users input values and see step-by-step computations
- **Data Visualizations** — Recharts-powered area charts and bar charts for sequence data
- **KaTeX Formula Rendering** — Beautiful mathematical typesetting with glassmorphism containers

- **Quiz System** — 7 questions per mission with immediate feedback, scoring, and pass/fail thresholds
- **Achievement Badges** — 8 unlockable achievements tracking progress across all missions
- **Persistent Progress** — localStorage-based tracking that survives page reloads
- **Sound Design** — Dual-mode audio engine (file-based + synthetic fallback) with 11 sound effects
- **Starfield Background** — Canvas-based animated background with twinkling stars, shooting stars, and nebulae
- **Logout Animation** — Dramatic rocket + comet collision explosion sequence

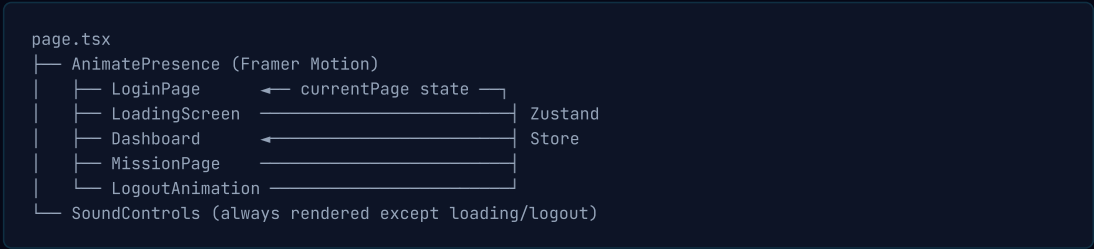
2. Technology Stack

CATEGORY	TECHNOLOGY	VERSION
Framework	Next.js (App Router)	^16.1.1
Language	TypeScript	^5
Runtime	Bun	latest
Styling	Tailwind CSS	^4
UI Components	shadcn/ui (New York style)	latest
Animation	Framer Motion	^12.23.2
State	Zustand	^5.0.6
Math Rendering	KaTeX	^0.16.46
Charts	Recharts	^2.15.4
Icons	Lucide React	^0.525.0
Fonts	Geist, Geist Mono, Orbitron, Electrolize	Google Fonts
Audio	Web Audio API	Browser Native
Toast	Sonner	^2.0.6
Forms	React Hook Form + Zod	latest
DB ORM	Prisma (SQLite)	^6.11.1

3. Architecture & Design

Single-Page Application Pattern

The entire application is a **single Next.js page** (`src/app/page.tsx`) that uses **Zustand-driven client-side routing**. There are no URL-based routes — all navigation happens through state transitions.



Data Flow

```
graph LR
    A[User Input] --> B[Zustand Store]
    B --> C[Component Re-render]
    C --> D[localStorage Persistence]
    B --> E[currentPage, username, progress, soundEnabled]
    E --> F[Math Algorithms]
    F --> G[Simulators]
    G --> H[Visualizations]
    H --> I[Quizzes]
    I --> J[Progress Updates]
```

4. Page Flow & User Journey

Login Page

- Starfield background with nebulae
- Radar SVG animation (rotating sweep)
- Telemetry panel (live stats)
- Scanline overlay effect
- Constellation dot pattern
- BGM starts on first interaction
- User enters callsign (username)
- Auth state machine: PENDING → VERIFYING → CONFIRMED

Loading Screen (Rocket Launch)

- 6 phases: Countdown (10→1), Ignition, Launch, Approach, Landing, Complete
- SVG rocket with animated fire/smoke
- Mission-specific planet landing
- Rocket launch sound effect

Dashboard (Mission Control)

- 6 Mission cards in responsive grid with planet images
- Progress indicator (quiz score) per mission
- Neon glow in mission color
- Achievements section with trophy icon
- Sound toggle & Logout buttons

Mission Page (8-Tab Content)

- Tab 1: OVERVIEW — Description of the sequence/algorithm
- Tab 2: FORMULA — KaTeX-rendered mathematical formula
- Tab 3: EXAMPLE — Step-by-step worked example
- Tab 4: SIMULATOR — Interactive computation with user input
- Tab 5: VISUALIZATION — Recharts area/bar charts
- Tab 6: APPLICATIONS — Real-world use cases
- Tab 7: HISTORY — Historical context and origins
- Tab 8: QUIZ — 7-question interactive quiz (70% pass threshold)

Logout Animation

- Rocket launches → Comet collision → Multi-ring explosion → Fade to black → Returns to login

5. The 6 Missions (Curriculum)

🌀 Mission 1: Collatz — The Unstoppable Sequence

Color	Cyan	Planet	planet-collatz.png
Topic	Collatz Conjecture	Formula	$f(n) = n/2$ if even; $3n+1$ if odd

Key Concept: Start with any positive integer. If even, divide by 2; if odd, multiply by 3 and add 1. The conjecture claims you always reach 1. Unproven despite verification up to 2^{68} .

Worked Example: Starting with $n=6$: $6 \rightarrow 3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ (8 steps)

🍌 Mission 2: Fibonacci — Nature's Golden Code

Color	Amber	Planet	planet-fibonacci.png
Topic	Fibonacci Sequence	Formula	$F_n = F_{n-1} + F_{n-2}$, $F_0=0$, $F_1=1$

Key Concept: Each number is the sum of the two preceding ones. Found throughout nature. Ratio converges to $\phi \approx 1.618$.

Worked Example: $F(0) \rightarrow F(9) = 0, 1, 1, 2, 3, 5, 8, 13, 21, 34$

🔺 Mission 3: Tribonacci — The Triple Helix

Color	Purple	Planet	planet-tribonacci.png
Topic	Tribonacci Sequence	Formula	$T_n = T_{n-1} + T_{n-2} + T_{n-3}$, $T_0=0$, $T_1=0$, $T_2=1$

Key Concept: Generalization of Fibonacci — sum of three preceding numbers. Ratio converges to tribonacci constant ≈ 1.839 .

Worked Example: $T(0) \rightarrow T(9) = 0, 0, 1, 1, 2, 4, 7, 13, 24, 44$

★ Mission 4: Lucas — The Parallel Universe

Color	Emerald	Planet	planet-lucas.png
Topic	Lucas Sequence	Formula	$L_n = L_{n-1} + L_{n-2}$, $L_0=2$, $L_1=1$

Key Concept: Fibonacci's twin with different seeds (2,1 instead of 0,1). Related: $L(n) = F(n-1) + F(n+1)$. Used in Lucas-Lehmer primality test.

Worked Example: $L(0) \rightarrow L(9) = 2, 1, 3, 4, 7, 11, 18, 29, 47, 76$

 Mission 5: Euclid — The Ancient Algorithm

Color	Rose	Planet	planet-euclidean.png
Topic	Euclidean Algorithm (GCD)	Formula	$\gcd(a,b) = \gcd(b, a \bmod b)$

Key Concept: The oldest algorithm still in use (~300 BC). Repeatedly replace larger number with remainder until 0. Essential for RSA cryptography.

Worked Example: $\text{GCD}(48,18)$: $48 \div 18 = 2r12 \rightarrow 18 \div 12 = 1r6 \rightarrow 12 \div 6 = 2r0 \rightarrow \text{GCD}=6$

 Mission 6: Division — The Foundation of Arithmetic

Color	Sky	Planet	planet-division.png
Topic	Division Algorithm (Theorem)	Formula	$a = bq + r$, where $0 \leq r < b $

Key Concept: Guarantees unique quotient q and remainder r for any integers a,b ($b \neq 0$). Foundation of modular arithmetic and number theory.

Worked Example: $43 = 7 \times 6 + 1$ ($q=6, r=1$, verified: $42+1=43$)

6. Component Breakdown

Custom Mission Components

COMPONENT	PURPOSE	KEY IMPLEMENTATION
StarfieldBackground	Animated canvas background	Canvas API, requestAnimationFrame, twinkling stars, shooting stars, nebula clouds
LoginPage	Sci-fi crew authentication	SVG radar, telemetry panel, scanline overlay, constellation dots, grunge inputs, auth state machine
LoadingScreen	Rocket launch transition	6-phase animation, SVG rocket with fire/smoke, screen shake, planet landing
Dashboard	Mission selection hub	6 mission cards with planet images, progress tracking, neon glow, achievements
MissionPage	Mission content viewer	8-tab layout, interactive simulator, Recharts visualizations, section progress
QuizComponent	Interactive quiz	One question at a time, immediate feedback, 70% pass threshold, celebration particles
KaTeXFormula	Math formula renderer	katex.render(), glassmorphism container, cyan glow border
LogoutAnimation	Logout transition	Rocket → comet collision → explosion → fade to black
SoundControls	Floating sound toggle	Fixed bottom-right, cyan glow, spring animation

shadcn/ui Components

44 pre-built components in New York style: accordion, alert, alert-dialog, aspect-ratio, avatar, badge, breadcrumb, button, calendar, card, carousel, chart, checkbox, collapsible, command, context-menu, dialog, drawer, dropdown-menu, form, hover-card, input, input-otp, label, menubar, navigation-menu, pagination, popover, progress, radio-group, resizable, scroll-area, select, separator, sheet, sidebar, skeleton, slider, sonner, switch, table, tabs, textarea, toast, toaster, toggle, toggle-group, tooltip

7. State Management

Zustand Store (src/store/appStore.ts)

The entire application state is managed through a single Zustand store:

```

interface AppState {
  // Navigation
  currentPage: 'login' | 'loading' | 'dashboard' | 'mission' | 'logout' | 'achievements';
  loadingTarget: AppPage;
  loadingMissionId: string | null;

  // User
  username: string;
  progress: UserProgress | null;

  // Mission
  currentMissionId: string | null;

  // Settings
  soundEnabled: boolean;

  // Actions
  setPage(page: AppPage): void;
  login(username: string): void;
  logout(): void;
  selectMission(missionId: string): void;
  setLoading(target: AppPage, missionId?: string | null): void;
  finishLoading(): void;
  updateProgress(missionId: string, update: {...}): void;
  setSoundEnabled(enabled: boolean): void;
  loadUserProgress(username: string): void;
}

```

State Transitions

```

login → LoginPage
└─ onAuthConfirmed → login(username), setLoading('dashboard')

loading → LoadingScreen
└─ onFinish → finishLoading() → navigates to loadingTarget

dashboard → Dashboard
├─ onMissionSelect → selectMission(id), setLoading('mission', id)
└─ onLogout → logout()

mission → MissionPage
├─ onBack → setLoading('dashboard')
└─ onQuizComplete → updateProgress()

logout → LogoutAnimation
└─ onComplete → setPage('login')

```

8. Sound & Audio Engine

SoundEngine Class (src/lib/soundEngine.ts)

A singleton class implementing dual-mode audio: **file-based playback** with **synthetic fallback**.

File-Based Sounds

SOUND	FILE	SYNTHETIC FALLBACK	USED BY
Background Music	/audio/bg.mp4	(none — silence)	Login, dashboard
Rocket Launch	/audio/rocketlaunch.mp4	Rising sawtooth + rumble	LoadingScreen
Rocket Crash	/audio/rocketcrash.mp4	Noise burst + low boom	LogoutAnimation
Deploy	/audio/deploy.mp3	C5-E5-G5 ascending	Success actions
Expand	/audio/expand.mp3	C5-E5-G5-C6 ascending	Achievement unlock
Fade	/audio/fade.mp3	G4-Eb4 descending	Failure/incorrect
Hover	/audio/hover.mp3	1200Hz sine blip	Button hover
Start	/audio/start.mp3	C5-E5-G5 ascending	Verification start
Transmission	/audio/transmission.mp3	C5-E5-G5-C6 ascending	Data transmission
Typing	/audio/typing.mp3	Random 600-1000Hz square	Keyboard input
Click	/audio/click.mp3	800Hz square blip	Button click

Synthetic Sound Generation

- **click()** — 800Hz, 0.08s, square wave
- **hover()** — 1200Hz, 0.04s, sine wave
- **type()** — 600-1000Hz random, 0.05s, square wave
- **success()** — C5 → E5 → G5 ascending arpeggio (sine)
- **failure()** — G4 → Eb4 descending (sawtooth)
- **achievement()** — C5 → E5 → G5 → C6 ascending arpeggio (sine)
- **launch()** — Rising sawtooth 80 → 400Hz + rumble triangle 40 → 60Hz
- **countdown()** — 440Hz, 0.15s, square wave
- **crash()** — Noise burst with lowpass filter sweep + 60 → 20Hz sine boom

Volume Control

- Default volume: 0.5
- BGM plays at 0.4× master volume
- Sound effects at 0.5-0.6× master volume
- Mute state pauses BGM and silences all effects

9. Math Algorithms Engine

File: src/lib/mathAlgorithms.ts

```
// Collatz
collatzSequence(start: number): number[]
// Returns: [6, 3, 10, 5, 16, 8, 4, 2, 1] for start=6

collatzSteps(start: number): { step: number; value: number; rule: string }[]
// Returns: Step-by-step with explanations like "6 is even → 6 ÷ 2 = 3"

// Fibonacci
fibonacciSequence(n: number): number[]
// Returns: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34] for n=10
fibonacciNth(n: number): number // Returns: 34 for n=10

// Tribonacci
tribonacciSequence(n: number): number[]
// Returns: [0, 0, 1, 1, 2, 4, 7, 13, 24, 44] for n=10
tribonacciNth(n: number): number // Returns: 44 for n=10

// Lucas
lucasSequence(n: number): number[]
// Returns: [2, 1, 3, 4, 7, 11, 18, 29, 47, 76] for n=10
lucasNth(n: number): number // Returns: 76 for n=10

// Euclidean Algorithm
euclideanAlgorithm(a: number, b: number): { gcd: number; steps: {...}[] }
// Returns: { gcd: 6, steps: [{a:48,b:18,q:2,r:12}, ...] }

// Division Algorithm
divisionAlgorithm(dividend: number, divisor: number): { quotient: number; remainder: number; steps: string[] }
// Returns: { quotient: 6, remainder: 1, steps: [...] }
```

10. Progress & Achievement System

Storage

- **Key:** `math-mission-control-progress`
- **Location:** Browser `localStorage`
- **Format:** JSON

8 Achievements

#	ID	NAME	ICON	CONDITION
1	<code>collatz-specialist</code>	Sequence Specialist		Pass Collatz quiz
2	<code>fibonacci-navigator</code>	Recursive Navigator		Pass Fibonacci quiz
3	<code>tribonacci-explorer</code>	Tribonacci Trailblazer		Pass Tribonacci quiz
4	<code>lucas-commander</code>	Lucas Commander		Pass Lucas quiz
5	<code>euclid-explorer</code>	Euclid Explorer		Pass Euclidean quiz
6	<code>division-commander</code>	Division Commander		Pass Division quiz
7	<code>math-astronaut</code>	Math Astronaut		Pass ALL quizzes
8	<code>quiz-master</code>	Quiz Master		100% on ALL quizzes

Achievement Unlocking Logic

1. When a quiz is passed ($\geq 70\%$), the corresponding mission achievement unlocks
2. When all 6 quizzes are passed, "Math Astronaut" unlocks
3. When all 6 quizzes score 100%, "Quiz Master" unlocks
4. Unlocks are timestamped and persisted to `localStorage`

11. Visual Design System

Color Palette

The theme uses **oklch** color space with a deep purple-blue base hue (270°) and cyan accent hue (195°):

VARIABLE	OKLCH VALUE	APPROXIMATE COLOR
<code>--background</code>	oklch(0.07 0.02 270)	Deep navy/black
<code>--foreground</code>	oklch(0.95 0.01 270)	Near white
<code>--card</code>	oklch(0.12 0.02 270)	Dark navy
<code>--primary</code>	oklch(0.75 0.15 195)	Cyan
<code>--secondary</code>	oklch(0.18 0.03 270)	Dark purple
<code>--destructive</code>	oklch(0.65 0.2 25)	Red-orange

Mission Colors

MISSION	TAILWIND COLOR	HEX APPROX	NEON GLOW
Collatz	cyan	#22d3ee	Cyan glow
Fibonacci	amber	#f59e0b	Amber glow
Tribonacci	purple	#a855f7	Purple glow
Lucas	emerald	#10b981	Green glow
Euclidean	rose	#f43f5e	Pink glow
Division	sky	#0ea5e9	Blue glow

Typography

FONT	USAGE	VARIABLE
Geist	Body text, UI elements	<code>--font-geist-sans</code>
Geist Mono	Code, monospace content	<code>--font-geist-mono</code>
Orbitron	Headings, mission titles, sci-fi text	<code>--font-orbitron</code>
Electrolize	Subheadings, data readouts	<code>--font-electrolize</code>

Design Effects

- Glassmorphism** — Semi-transparent backgrounds with blur backdrop
- Neon Glow** — Box-shadow with mission-colored light
- Scanlines** — CSS-animated overlay for retro-CRT effect
- Starfield** — Canvas-based animated background
- Screen Shake** — CSS transform on rocket launch
- Custom Scrollbar** — Cyan-tinted scrollbar matching the theme

12. Asset Library

Audio Files (public/audio/)

FILE	FORMAT	PURPOSE
bg.mp4	MP4	Looping background music
rocketlaunch.mp4	MP4	Rocket launch sound effect
rocketcrash.mp4	MP4	Rocket crash/explosion
click.mp3	MP3	UI click feedback
deploy.mp3	MP3	Deploy/success sound
expand.mp3	MP3	Achievement unlock
fade.mp3	MP3	Failure/incorrect
hover.mp3	MP3	Hover feedback
start.mp3	MP3	Verification start
transmission.mp3	MP3	Data transmission
typing.mp3	MP3	Keyboard typing

Planet Images (public/planets/)

FILE	MISSION	THEME COLOR
planet-collatz.png	Mission Collatz	Cyan
planet-fibonacci.png	Mission Fibonacci	Amber
planet-tribonacci.png	Mission Tribonacci	Purple
planet-lucas.png	Mission Lucas	Emerald
planet-euclidean.png	Mission Euclid	Rose
planet-division.png	Mission Division	Sky

13. File Structure

```
/home/z/my-project/ | ├── Configuration Files | ├── Caddyfile, bun.lock, components.json | ├──
eslint.config.mjs, next.config.ts, package.json | ├── postcss.config.mjs, tailwind.config.ts,
tsconfig.json | ├── prisma/ | └── schema.prisma (User + Post models, unused in app) | ├── db/ | └──
custom.db (SQLite database, unused in app) | ├── public/ | ├── logo.svg, robots.txt | ├── audio/ (11
sound files: bg.mp4, rocketlaunch.mp4, etc.) | └── planets/ (6 planet PNGs) | ├── src/ | ├── app/ | |
├── globals.css (Dark cosmic theme + animations) | | ├── layout.tsx (Root layout, 4 fonts, dark mode,
Toaster) | | ├── page.tsx (Main SPA with AnimatePresence routing) | | └── api/route.ts (Minimal GET
API) | | | ├── components/ | | | ├── mission/ (9 custom components) | | | ├── StarfieldBackground.tsx |
| | ├── LoginPage.tsx | | | ├── LoadingScreen.tsx | | | ├── Dashboard.tsx | | | └── MissionPage.tsx |
```

```

| | | └─ QuizComponent.tsx | | | └─ KaTeXFormula.tsx | | | └─ LogoutAnimation.tsx | | | └─
SoundControls.tsx | | | └─ ui/ (44 shadcn/ui components) | | | └─ hooks/ (use-mobile.ts, use-toast.ts)
| | | └─ lib/ | | | └─ db.ts (Prisma client singleton) | | | └─ mathAlgorithms.ts (6 algorithm
functions) | | | └─ missionData.ts (6 mission data objects) | | | └─ progressTracker.ts (Progress +
achievements logic) | | | └─ soundEngine.ts (Web Audio API engine) | | | └─ utils.ts (cn() utility) | | |
└─ store/ | └─ appStore.ts (Zustand global store) | └─ examples/websocket/ (Socket.IO example) └─
agent-ctx/ (Agent documentation) └─ download/ (Generated files)

```

14. Configuration

CONFIG	KEY SETTINGS
Next.js	output: "standalone", TS errors ignored, strictMode off
Tailwind	Dark mode "class", custom CSS vars, tailwindcss-animate
TypeScript	Strict mode, @/* → ./src/* alias, bundler resolution
ESLint	Flat config, core-web-vitals + TS, most rules relaxed

15. How to Run

```

# Install dependencies
bun install

# Start development server (port 3000)
bun run dev

# Run linter
bun run lint

# Database (if needed)
bun run db:push
bun run db:generate

# Production build
bun run build
bun run start

```

Summary

Mathematical Mission Control is a fully self-contained, client-side educational web application that combines:

- 🎮 **Gamification** — Missions, quizzes, achievements, progress tracking
- 🎨 **Immersive Design** — Space theme with animations, sound effects, and particle systems
- 📐 **Real Mathematics** — Working algorithms, KaTeX formulas, interactive simulators, data visualizations
- 🔧 **Modern Stack** — Next.js 16, TypeScript, Tailwind CSS, Zustand, Framer Motion
- 🔊 **Rich Audio** — 11 sound effects with synthetic fallbacks and background music
- 💾 **Persistent Progress** — localStorage-based saving with achievement tracking

The application teaches 6 core mathematical concepts through a narrative of space exploration, making abstract mathematics tangible and engaging through interactive computation, visual

feedback, and rewarding progression.

MATHEMATICAL MISSION CONTROL – SYSTEM DOCUMENTATION